

Doc-Genie: Python Function Docstring Generator

1. Introduction

Doc-Genie is an intelligent Python documentation generator designed to automatically create professional docstrings for Python functions. Many software projects suffer from poor or missing documentation, making maintenance and collaboration difficult. Doc-Genie solves this problem by analyzing Python code using Abstract Syntax Tree (AST) techniques and automatically generating well-structured documentation in standard formats such as Google style and NumPy style.

The system reads Python functions, understands their parameters, return values, and internal logic, and produces accurate documentation. It also provides an easy-to-use web interface using Gradio, allowing developers, students, and teams to generate documentation quickly.

2. Objectives of the Project

- Automatically generate Python docstrings from source code.
- Analyze function structure using AST (Abstract Syntax Tree).
- Support multiple documentation styles such as Google and NumPy.
- Provide a simple graphical interface using Gradio.
- Allow export of documentation as PDF or text files.

3. Core Technologies Used

- Python 3.8+: Core programming language used for system development.
- AST (Abstract Syntax Tree): Used for parsing and analyzing Python code structure.
- Gradio: Used to create the web interface for user interaction.
- ReportLab: Used for generating PDF documentation reports.
- Regular Expressions: Used for pattern detection in code.

4. System Architecture

The Doc-Genie system consists of multiple modules working together. The user provides Python code through the interface. The system parses the code using AST and extracts function information such as name, parameters, default values, and return types. The logic analyzer examines the function body to detect loops, conditions, operations, and return statements. Finally, the docstring generator creates formatted documentation based on the selected style.

5. Workflow of the System

1. User enters Python code or uploads a .py file.
2. AST parser analyzes the code structure.
3. Function signatures and parameters are extracted.
4. Logic analysis detects operations, loops, and conditions.
5. Docstring generator creates documentation.
6. User can export the documentation as PDF or TXT.

6. Core Coding Implementation

Example Python code used to extract function signatures using AST:

```
import ast

def extract_function_signature(code):
    tree = ast.parse(code)
    for node in ast.walk(tree):
        if isinstance(node, ast.FunctionDef):
            function_name = node.name
            parameters = [arg.arg for arg in node.args.args]
            return {
                "name": function_name,
                "parameters": parameters
            }
```

Example code for generating a simple Google-style docstring:

```
def generate_google_docstring(name, params):
    docstring = f'"""{name} function description.'

    docstring += "\n\nArgs:\n"
    for p in params:
        docstring += f"    {p} (type): Description of {p}.\n"

    docstring += "\nReturns:\n"
    docstring += "    type: Description of return value.\n"
    docstring += '"""'

    return docstring
```

7. Example Input and Output

Example Input Function:

```
def factorial(n):  
    if n == 0:  
        return 1  
    return n * factorial(n-1)
```

Generated Docstring Output:

```
"""Calculate factorial of a number.
```

Args:

```
    n (int): Input number.
```

Returns:

```
    int: Factorial value of the number.
```

```
"""
```

8. Applications of Doc-Genie

- Automatic documentation for large Python codebases.
- Generating API documentation during development.
- Helping students learn proper Python documentation.
- Improving code readability and maintainability.
- Supporting code review and quality assurance processes.

9. Advantages

- Saves time by automating documentation.
- Ensures consistent documentation style.
- Improves code readability and maintainability.
- Easy to use through web interface.
- Supports export and sharing features.

10. Future Enhancements

In the future, Doc-Genie can be extended to support additional programming languages such as Java, C++, and JavaScript. Integration with IDEs like Visual Studio Code and

PyCharm can provide real-time documentation generation. Machine learning techniques can also be used to generate more intelligent and detailed descriptions of functions.

11. Conclusion

Doc-Genie provides an effective solution for automated Python documentation generation. By combining AST analysis, intelligent logic detection, and a user-friendly interface, the system simplifies the documentation process for developers and students. This project demonstrates how automation can improve software development practices and ensure well-documented code.